



Bliss Rent

Phase 3 — Admin Dashboard & Operations

Completion Report · Dubai-only car rental platform · Prepared 2026-08-25

Build: PASS

Lint: PASS (0 errors)

Tests: 120/120 PASS

RLS suite: re-verified

Scope: Phase 3 only

1 What was built

The full staff-facing admin dashboard for Bliss Rent, covering all ten areas named in the Phase 3 brief: dashboard overview, bookings, vehicles/fleet, vehicle availability, customers, driver information, payments, complaints/support, pricing, and admin settings. It is built entirely on top of the Phase 0 `admin_profiles` / `is_admin()` / Row Level Security foundation and reuses the Phase 1 availability engine and Phase 2 pricing/payment logic rather than duplicating any of it — the recurring theme across every section below is "read/write through the existing RLS-governed tables," not a second implementation of anything that already existed.

Nothing outside Phase 3 was touched: no WhatsApp Business API integration, no production deployment, no advanced analytics, no automated marketing, no other-emirates support, and no new business features were started.

2 Admin routes

Route	Page	Guarded?
<code>/admin/login</code>	AdminLoginPage	Public (redirects away if already signed in as an admin)
<code>/admin</code>	DashboardPage	AdminRoute
<code>/admin/bookings</code>	BookingsListPage	AdminRoute
<code>/admin/bookings/:id</code>	BookingDetailPage	AdminRoute
<code>/admin/fleet</code>	FleetListPage	AdminRoute
<code>/admin/fleet/new</code>	VehicleFormPage (add mode)	AdminRoute
<code>/admin/fleet/:id</code>	VehicleFormPage (edit mode)	AdminRoute
<code>/admin/availability</code>	AvailabilityCalendarPage	AdminRoute
<code>/admin/customers</code>	CustomersListPage	AdminRoute
<code>/admin/customers/:id</code>	CustomerDetailPage	AdminRoute
<code>/admin/payments</code>	PaymentsListPage	AdminRoute
<code>/admin/complaints</code>	ComplaintsListPage	AdminRoute
<code>/admin/complaints/:id</code>	ComplaintDetailPage	AdminRoute
<code>/admin/pricing</code>	PricingManagementPage	AdminRoute
<code>/admin/audit-log</code>	AuditLogPage	AdminRoute
<code>/admin/settings</code>	AdminSettingsPage	AdminRoute

All routes are wired into `src/App.tsx`, wrapped by `AdminAuthProvider`. Every route except `/admin/login` renders inside `<AdminRoute><AdminLayout /></AdminRoute>`, so there is exactly one place in the codebase that decides whether a visitor may see admin content.

3 Admin components

Core / shell

- AdminAuthContext.tsx** — loads the Supabase session, then looks up the matching `admin_profiles` row. Exposes `{loading, session, adminProfile, notAuthorized, signOut, refreshProfile}`.
- AdminRoute.tsx** — the single route guard: spinner while loading, redirect to `/admin/login` if no session or no admin profile, otherwise render.
- AdminLoginPage.tsx** — email/password sign-in via `supabase.auth.signInWithPassword()`; shows a clear "not authorized" message (with a sign-out-and-retry link) for a real account that simply isn't staff.
- AdminLayout.tsx** — desktop sidebar + mobile drawer navigation (Dashboard / Bookings / Fleet / Availability / Customers / Payments / Complaints / Pricing / Audit Log / Settings), header with language switcher, `<Outlet/>` for nested routes.
- adminApi.ts** — `AdminApiError` (shared error class used by every admin*Api.ts file) plus dashboard KPI/recent-activity queries.

Shared UI

- AdminPageHeader** — title/description/action-slot header used on every page.
- AdminTabs** — generic, typed tabbed filter with optional counts.
- AdminStatusBadge** — one status-chip component covering every booking/payment/complaint/vehicle/operational-status value, colour-coded by tone.

Full per-feature component/API breakdown is in sections 4–11 below.

4 Booking management

Files: `bookings/adminBookingsApi.ts` , `bookings/BookingsListPage.tsx` , `bookings/BookingDetailPage.tsx` .

List page: six tabs (All / Pending / Confirmed / Active / Completed / Cancelled), free-text search across customer name/email, vehicle make/model, and booking id, a table with customer, vehicle, dates, amount, payment status, and booking status columns, and a link into the detail page.

Detail page: full booking / customer / driver / rental / payment / status-history sections, plus a status-change dropdown. Changing status runs a plain RLS-governed `UPDATE` — no new Edge Function — because the existing "admins manage bookings" policy from Phase 0 already covers it, and the (Phase 3-extended) `handle_booking_status_change()` trigger automatically writes both a `booking_status_history` row and an `audit_logs` row from that single update.

Private driver documents (license/ID) are only ever exposed as 60-second signed URLs from the private `driver-documents` bucket, fetched on demand — never a public URL, never embedded directly in the page.

5 Fleet management

Files: `fleet/adminFleetApi.ts` , `fleet/vehicleValidation.ts` , `fleet/FleetListPage.tsx` , `fleet/VehicleFormPage.tsx` .

List page: six tabs (All / Available / Reserved / Rented / Maintenance / Unavailable) driven by the new `vehicle_operational_status` view, a photo-card grid per vehicle.

Add/Edit page (one component, mode decided by the presence of a route `:id`): category, make, model, year, transmission, seats, plate number, and status fields, with client-side validation mirroring the schema's own constraints; an inline image manager (upload, set-primary, remove) against the public `vehicle-images` bucket. Every write is a plain RLS-governed insert/update/storage call — no Edge Function needed — and the new `vehicles_audit` trigger logs creation, updates, and status changes automatically.

6 Availability management

Files: `availability/adminAvailabilityApi.ts` , `availability/calendarGrid.ts` , `availability/AvailabilityCalendarPage.tsx` .

A per-vehicle month calendar showing real bookings (confirmed/active), colour-coded by status, plus a maintenance/retired banner when applicable. This deliberately reads real `bookings` rows directly — it is **not a second availability engine**. The Phase 1 `available_vehicles(start_date, end_date)` function remains the only place that answers "is this vehicle bookable for date range X," and nothing in this page recomputes that logic.

7 Customer management

Files: `customers/adminCustomersApi.ts` , `customers/CustomersListPage.tsx` , `customers/CustomerDetailPage.tsx` .

List page: All Customers / Active Renters tabs, search by name/email/phone, booking counts computed from real booking rows (never invented). Detail page: contact info, current booking, and full booking history, with no unnecessary private information surfaced.

8 Driver-information management

Driver details are shown exactly where the spec requires — inside **Booking Detail** (section 4/6 of the admin UI) — as information the customer supplied for that specific booking: full name, date of birth, license number/country/expiry, and signed-URL links to their license/ID documents. There is no company-driver table, no driver assignment UI, no driver availability or dispatch logic, and no driver marketplace anywhere in the codebase. This matches the business rule that the company never provides drivers.

9 Payment management

Files: `payments/adminPaymentsApi.ts` , `payments/PaymentsListPage.tsx` .

Five tabs (All / Pending / Paid / Failed / Refunded), a table with customer, booking link, amount, provider reference (or a "Test mode" label when the TEST-ONLY provider from Phase 2 was used), date, and status. This page is **strictly read-only** by design — it exposes no update/write function at all. Phase 2's `confirm-payment` Edge Function and TEST payment provider remain the only writers of payment state, per the brief's explicit "do NOT create a second payment system." A dedicated test (`adminPaymentsApi.test.ts`) asserts the module's only export is `fetchPayments` .

10 Complaint management

Files: `complaints/adminComplaintsApi.ts` , `complaints/ComplaintsListPage.tsx` , `complaints/ComplaintDetailPage.tsx` .

List page: Open / In Progress / Resolved / Closed / All tabs. Detail page shows complaint id, customer, linked booking, date, and description, plus an editable status dropdown, internal notes (staff-only), and a resolution summary field. Saving stamps `resolved_at` automatically when status moves into resolved/closed, and clears it if reopened. The structure (status + notes + resolution, linked to a booking and customer) is intentionally shaped so a future WhatsApp Business API integration has a natural home to write into — but no WhatsApp integration itself was built, per the explicit stop condition.

11 Pricing management

Files: `pricing/adminPricingApi.ts`, `pricing/pricingValidation.ts`, `pricing/PricingManagementPage.tsx`.

A vehicle picker plus one form covering the full daily/weekly/monthly/3-month ladder for that vehicle, reusing `TERM_LABELS` from the existing `src/lib/pricing.ts` — there is exactly one pricing-calculation implementation in the codebase (`quoteForDays` / `resolveTermForDays`), and this page never recomputes or duplicates it, only edits the input prices it reads from. Client-side validation mirrors the new database constraint (client price cannot exceed list price) before the row is even submitted. Saving upserts only the terms the admin actually filled in, keyed on the existing (`vehicle_id`, `term`) unique constraint.

12 Audit logging

Files: `auditlog/adminAuditApi.ts`, `auditlog/AuditLogPage.tsx`, plus the trigger functions in the Phase 3 migration.

A read-only, filterable feed (All / Fleet / Pricing / Bookings / Complaints) over the `audit_logs` table, showing action, entity, actor, and timestamp for the most recent 50 rows. Nothing in the frontend writes to `audit_logs` directly — every row is produced by a database trigger:

- `vehicles_audit` — vehicle created / updated / status changed
- `pricing_audit` — pricing created / changed
- `complaints_audit` — complaint status changed
- `handle_booking_status_change()` (Phase 0 trigger, extended in place) — booking status changed

This is one cross-entity activity feed layered on top of the existing per-booking `booking_status_history` table, not a second, parallel logging mechanism.

13 Security / RLS changes

- **No new client-facing auth system.** Admin access is Supabase Auth plus a matching `admin_profiles` row — the exact Phase 0 model, unchanged. Admin accounts are still provisioned by a one-time manual SQL insert (see `docs/SETUP.md`); there is no self-serve admin sign-up UI.
- **Every admin write goes through RLS, not a bypass.** All `admin*Api.ts` files use the same anon-key `supabase` client as the rest of the app. Nothing uses the service-role key from the frontend.
- **New, narrowly-scoped policy:** `"admins insert audit logs"` — admins may insert rows attributed to themselves (`actor_id = auth.uid()`) or system-attributed (`actor_id is null`). This exists only because the audit triggers run as the acting admin, not a privileged role, and would otherwise be blocked by RLS. There is still no general client write path to `audit_logs`.
- **vehicle_operational_status is a `security_invoker = true` view** — it enforces RLS as the querying admin, not the view owner, closing the classic "a plain view silently bypasses RLS" gotcha.
- **Driver/customer documents remain private** — Phase 0's storage policies (customer writes their own booking's folder, admin-only read) are untouched; the admin UI only ever requests short-lived signed URLs.
- **Payments still have no client write path** — confirmed unchanged; the admin Payments page is read-only.

14 Database migrations

New file: `supabase/migrations/20260827000000_phase3_admin_dashboard.sql` (220 lines). Summary of contents:

```
1. admins insert audit logs      - new INSERT policy on audit_logs
2. audit_vehicle_change()       - trigger fn + trigger on vehicles
   audit_pricing_change()       - trigger fn + trigger on pricing
   audit_complaint_status_change() - trigger fn + trigger on complaints
   handle_booking_status_change() - EXISTING Phase 0 trigger fn, replaced
                                   in place to also write audit_logs
2b. complaints.internal_notes (text, nullable)
    complaints.resolution      (text, nullable)
3. pricing.client_price_le_list_price CHECK (client_price <= list_price)
4. vehicle_operational_status VIEW (security_invoker = true)
   grant select ... to authenticated
5. New indexes: payments_status_idx, complaints_status_idx,
   audit_logs_created_at_idx, bookings_created_at_idx
```

Also updated: `supabase/stub_supabase_platform.sql` — the LOCAL validation stub only (not a change to the real Supabase project). Three genuine gaps were found and fixed while testing this migration against a from-scratch local Postgres 16 database:

1. `grant usage on schema auth ...; grant execute on function auth.uid() ...;` — without this, any direct table write under the `authenticated` role that referenced `auth.uid()` (including the pre-existing Phase 0 booking-status trigger) failed with "permission denied for schema auth" locally, even though it works fine against real Supabase.
2. `alter table storage.objects enable row level security;` — without this, every Phase 0 storage policy existed but was never actually enforced locally, so the "customer cannot read driver-documents" test silently passed for the wrong reason.
3. Baseline table/sequence grants (`grant ... to anon, authenticated, service_role` + matching `alter default privileges`) — real Supabase grants these by default; the stub hadn't, so a plain `select` under `authenticated` failed with "permission denied for table X" before RLS even got a chance to run.

These are durable improvements to the test tooling used across every phase, not scope creep — they don't change the real schema or its behaviour on the hosted Supabase project at all.

15 Tests

New automated tests — 42 tests across 9 new files

File	Covers	Tests
AdminRoute.test.tsx	Admin authentication + unauthorized-user-blocked	4
fleet/vehicleValidation.test.ts	Vehicle form validation	6
fleet/adminFleetApi.test.ts	Vehicle listing, CRUD, status changes	6
bookings/adminBookingsApi.test.ts	Booking listing, booking detail, status update, private document access (signed URL)	6
customers/adminCustomersApi.test.ts	Customer listing (with real booking-count aggregation)	3
payments/adminPaymentsApi.test.ts	Payment listing + read-only enforcement	3
complaints/adminComplaintsApi.test.ts	Complaint management (status/notes/resolution, resolved_at stamping)	3
pricing/pricingValidation.test.ts	Pricing validation (mirrors the DB check constraint)	5
pricing/adminPricingApi.test.ts	Pricing management (draft-building + ladder upsert)	3
availability/calendarGrid.test.ts	Calendar layout math	3

API-level tests mock the Supabase client (`src/test/supabaseMock.ts` , a minimal chainable stand-in) so each `admin*Api.ts` function's query shape and return-value logic is verified without a live database. **RLS/security-boundary enforcement itself cannot be meaningfully exercised by Vitest** — that requires a real Postgres session evaluating real policies — so it is verified separately, against an actual local Postgres 16 instance, as detailed in section 18 below and re-confirmed the same day this report was produced.

Full suite result

`npm run test` — **21 test files, 120 tests, all passing** (78 pre-existing Phase 0–2 tests + 42 new Phase 3 tests; nothing pre-existing was broken).

16 Build result

`npm run build ( tsc -b && vite build )` — **PASS**, zero TypeScript errors. One pre-existing informational warning from Vite about the main JS chunk exceeding 500 kB after minification (not a Phase 3 regression, not an error, and not something the brief asked to be addressed — noted for awareness only).

One real bug was found and fixed during this pass: two admin pages (`BookingsListPage.tsx` , `BookingDetailPage.tsx`) imported `AdminApiError` from `adminBookingsApi.ts` , which never re-exported it (it only imports it from `adminApi.ts` for its own internal use). Fixed by importing `AdminApiError` directly from `adminApi.ts` in both consumer files, matching the pattern used everywhere else.

17 Lint result

`npm run lint (oxlint)` — **PASS**, exit code 0, zero errors. 11 warnings, all of the pre-existing `react(set-state-in-effect)` / `react(only-export-components)` kind already present throughout the Phase 1–2 codebase's "fetch on mount" pattern (confirmed by diffing lint output against the pre-Phase-3 commit: 9 such warnings existed before this phase, matching the same pattern now appearing 11 times across the larger admin codebase). No new category of lint issue was introduced.

18 Manual QA performed

Database / RLS (fully performed, against a real Postgres instance): dropped and recreated a local `bliss_phase3_test` database, reapplied `stub_supabase_platform.sql` and all four migrations in order, then re-ran the full Phase 3 RLS test script covering:

- A non-admin customer sees only their own booking, cannot write to admin-only tables, cannot forge `audit_logs` rows, cannot read other admins' profiles, and cannot read `driver-documents` storage objects (not even their own — read is admin-only).
- An admin sees all bookings, can update a vehicle (triggering a correctly-attributed `audit_logs` row), can insert valid pricing (triggering a correctly-shaped audit row), is correctly blocked by the new check constraint when client price exceeds list price, can update booking status (triggering both a `booking_status_history` row and an `audit_logs` row from the one update), and can read `driver-documents` .
- `vehicle_operational_status` correctly classifies a vehicle with a future confirmed booking as `reserved` , and correctly reports `maintenance` once the vehicle's own status is set to maintenance (overriding the booking-derived state).

Every check above passed on this final re-verification run, performed the same day as this report.

Code-level review (performed): full route tree in `src/App.tsx` reviewed end-to-end against the spec's required navigation (login → dashboard → bookings → booking detail → fleet → add/edit vehicle → availability → customers → payments → complaints → pricing → audit log → settings → logout); every admin page's translation keys cross-checked against the new `admin.*` namespace added to both `en.ts` and `ar.ts` (mirrored key-for-key, RTL-safe `start-` / `end-` classes reused from the existing site-wide pattern); a local Vite dev server was started and its routes smoke-checked for basic reachability.

What could not be performed from this environment: an actual click-through against the live, hosted Supabase project (real admin login → real data in every tab → language toggle observed live → sign-out) requires a browser session authenticated against your real project, which this sandboxed environment does not have network/browser access to. The Claude-in-Chrome browser bridge was attempted and reported "not connected" in this session. Section 21 below lists the exact steps to run that live walkthrough yourself.

19 Files changed

55 files changed — 5,014 insertions, 29 deletions. New files (45), modified files (10):

Area	New files
Core / routing	AdminAuthContext.tsx , AdminLayout.tsx , AdminLoginPage.tsx , AdminRoute.tsx (+ .test.tsx), adminApi.ts
Shared UI	shared/AdminPageHeader.tsx , shared/AdminStatusBadge.tsx , shared/AdminTabs.tsx
Dashboard	dashboard/DashboardPage.tsx
Bookings	bookings/adminBookingsApi.ts (+ .test.ts), bookings/BookingsListPage.tsx , bookings/BookingDetailPage.tsx
Fleet	fleet/adminFleetApi.ts (+ .test.ts), fleet/vehicleValidation.ts (+ .test.ts), fleet/FleetListPage.tsx , fleet/VehicleFormPage.tsx
Availability	availability/adminAvailabilityApi.ts , availability/calendarGrid.ts (+ .test.ts), availability/AvailabilityCalendarPage.tsx
Customers	customers/adminCustomersApi.ts (+ .test.ts), customers/CustomersListPage.tsx , customers/CustomerDetailPage.tsx
Payments	payments/adminPaymentsApi.ts (+ .test.ts), payments/PaymentsListPage.tsx
Complaints	complaints/adminComplaintsApi.ts (+ .test.ts), complaints/ComplaintsListPage.tsx , complaints/ComplaintDetailPage.tsx
Pricing	pricing/adminPricingApi.ts (+ .test.ts), pricing/pricingValidation.ts (+ .test.ts), pricing/PricingManagementPage.tsx
Audit log	auditlog/adminAuditApi.ts , auditlog/AuditLogPage.tsx
Settings	settings/adminSettingsApi.ts , settings/AdminSettingsPage.tsx
Test infra	src/test/supabaseMock.ts
Database	supabase/migrations/20260827000000_phase3_admin_dashboard.sql

Modified files	What changed
src/App.tsx	Added AdminAuthProvider + full /admin/* route tree
src/types/database.ts	Added vehicle_operational_status view type; complaints.internal_notes / resolution
src/types/domain.ts	Added AdminProfile , AdminBookingWithDetails , AdminCustomerWithStats , AdminVehicleWithDetails , AdminPaymentWithDetails , AdminComplaintWithDetails , AdminAuditLogEntry , VehicleDraft , PricingDraft , and related types
src/i18n/locales/en.ts / ar.ts	Full admin.* namespace, mirrored key-for-key (271 lines added to each)
supabase/stub_supabase_platform.sql	Three local-validation-only fixes (auth schema grants, storage RLS enforcement, baseline grants) — see section 14
docs/ARCHITECTURE.md	New "Phase 3 — admin dashboard & operations" section
docs/DATABASE.md	Documents the new view, triggers, columns, constraint, indexes
src/features/admin/README.md	Replaced the Phase 0 placeholder with the real Phase 3 file map

20 Unresolved issues

None outstanding in the delivered code. Two deliberate, documented scope boundaries (not defects) worth restating:

- **Payments admin page is read-only.** This is intentional per the brief ("do NOT create a second payment system"), not a missing feature — payment state can only change through Phase 2's confirm-payment Edge Function / TEST provider.
- **Live browser QA against the hosted Supabase project was not possible from this sandboxed environment** (no network/browser access to your real project or its data). Everything that could be verified without that access — build, lint, the full automated test suite, and the complete RLS/security-boundary suite against a real local Postgres instance — was verified and passed. See section 21 for exactly what to check yourself.

21 Exact manual steps required

Action needed — before or during your own walkthrough:

1. If you haven't already, create at least one admin account by inserting a row into `admin_profiles` for a real Supabase Auth user, via the Supabase SQL Editor — this has been a manual step since Phase 0 (see `docs/SETUP.md`, "Creating your first admin user"). There is still no self-serve admin sign-up UI, by design.
2. Run the Phase 3 migration against your live Supabase project (via the Supabase CLI or the SQL Editor):
`supabase/migrations/20260827000000_phase3_admin_dashboard.sql`. It has already been validated end-to-end against a local Postgres 16 clone of your schema; running it on the real project should be a clean apply.
3. `npm install && npm run dev` locally (or deploy), then walk through: `/admin/login` → sign in with your admin account → Dashboard → Bookings (open a booking, change its status, confirm it appears in Booking Detail's status history) → Fleet (add a vehicle, upload an image, change its status) → Availability (pick a vehicle, confirm bookings render on the calendar) → Customers → Payments (confirm it is read-only) → Complaints (open one, add internal notes/resolution, mark resolved) → Pricing (edit a vehicle's ladder, confirm a client price higher than list price is rejected) → Audit Log (confirm your actions above appear) → Settings → switch language to Arabic and confirm the whole dashboard mirrors to RTL correctly → Sign out.
4. Separately, open the site in a private/incognito window signed in as an ordinary customer (or logged out) and confirm every `/admin/*` URL redirects to `/admin/login` rather than showing any admin content.

Phase 3 complete. Stopping here.

Per the Phase 3 brief's explicit stop condition, no work on WhatsApp Business API integration, production deployment, advanced analytics, automated marketing, other-emirates support, or new business features has been started. Waiting for your explicit confirmation before beginning Phase 4.